Instituto Superior de Engenharia
Politécnico de Coimbra

Introdução à Inteligência Artificial

Relatório Trabalho Prático nº 2 - Problema de Otimização

Departamento de Engenharia Informática e de Sistemas (DEIS)

Luana Senra Rabello - 2024129510
Thuane Vitória Vieira de Lyra - 2024144204

Coimbra
2025

1. Introdução

Neste trabalho prático teve-se como objetivo conceber, implementar e analisar métodos de otimização que encontrem boas soluções para o problema do roteiro turístico. Ao longo do relatório analisa-se as diferenças entre as soluções encontradas pelo Algoritmo de Pesquisa Local (Trepa-Colinas), o Algoritmo Evolutivo e o Algoritmo Híbrido.

O problema estudado consiste em selecionar uma quantidade **m** de pontos turísticos dentre um número **C** de candidatos para serem construídos numa localidade, de modo a que a distância entre cada par seja a maior possível, ou seja, o objetivo é que este valor seja maximizado.

Dado esse contexto, a função de avaliação (fitness) constrói-se no princípio do cálculo da média dos custos de cada par de elementos que foram selecionados, ou seja, que têm o bit a 1. Caso a quantidade de elementos selecionados não seja exatamente igual à quantidade **m** imposta anteriormente, a função é encerrada zerando o valor da soma (penalização). Considera-se, nesse caso, que a solução proposta é inválida. Por outro lado, se as condições forem cumpridas, a solução é válida.

2. Descrição dos algoritmos

2.1. Algoritmo de Pesquisa Local

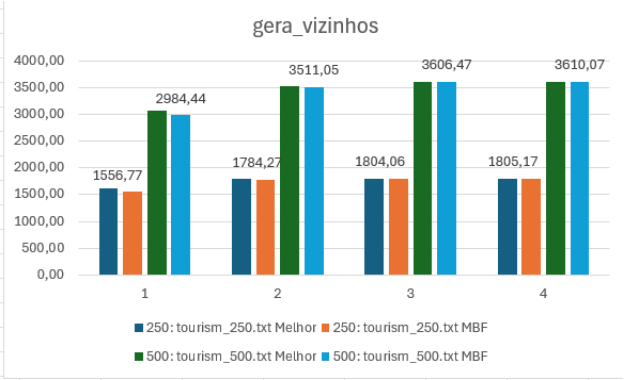
O algoritmo de pesquisa local escolhido, Trepa-Colinas, funciona como um alpinista escalando uma montanha pois, analogamente a isso, o algoritmo busca sempre aumentar a qualidade da solução. Inicia-se com uma solução aleatória e, a partir disso, gera uma vizinhança de possíveis soluções e calcula o fitness de cada uma, guardando apenas o maior valor. Para uma melhor análise experimental, foram implementadas duas versões da função que gera a vizinhança - uma delas (*gera_vizinho),* realiza uma troca simples entre um local ativo com bit a 1 e um local inativo com bit a 0; e a outra (*gera_vizinho2)* faz uma troca dupla simultânea, que permite saltos maiores no espaço de pesquisa. Ao utilizar as funções *gera_vizinho* e *gera_vizinho2*, garante-se matematicamente que o número de locais selecionados (**m**) permanece constante em cada iteração. Desta forma, o Trepa-Colinas navega sempre dentro do espaço de soluções válidas, poupando o custo computacional de corrigir soluções, não precisando assim de utilizar reparação nem penalização. O maior problema deste algoritmo de pesquisa local é que ele pode ficar preso em um máximo local e não global, o que não reflete a melhor das soluções na vizinhança.

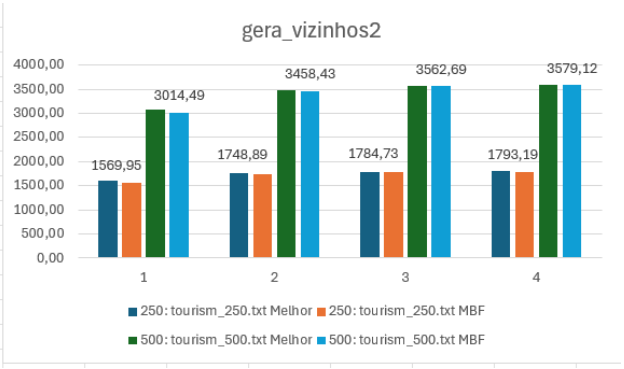
2.1.1 Testes experimentais da pesquisa local:

A análise foca-se nas instâncias de maior complexidade (*tourism_250.txt* e *tourism_500.txt*), onde a capacidade de escapar a ótimos locais é crítica.

2.1.2. Comparação de Vizinhanças (*gera_vizinho* VS *gera_vizinho2*)

Para avaliar o impacto da estratégia de movimento no espaço de procura, comparou-se a Vizinhança 1 (Troca Simples) com a Vizinhança 2 (Troca Dupla).



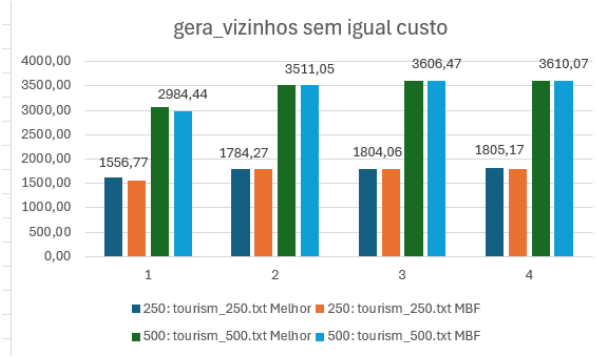


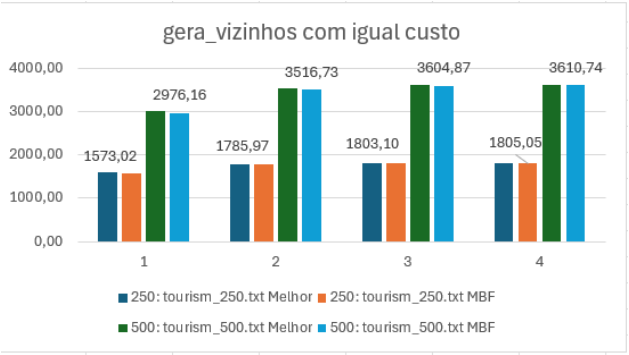
Após a análise detalhada dos gráficos, chega-se à conclusão de que o Trepa-Colinas com a função *gera_vizinho* obtém melhores soluções quando comparado ao Trepa-Colinas ao utilizar a função *gera_vizinho2.*

2.1.3. Critério de Aceitação (Igual Custo)

Em seguida, avaliou-se o impacto de aceitar soluções de igual custo (>=) *versus* aceitar apenas melhorias estritas (>).

O Trepa-Colinas neste experimento estará a utilizar a função *gera_vizinho.*





Neste experimento, nota-se que curiosamente ocorre um "empate": 4 dos MBF daqueles com igual custo são melhores do que os sem igual custo e 4 daqueles sem igual custo são melhores dos que consideram igual custo.

2.2. Algoritmo evolutivo

O algoritmo Evolutivo pode ser comparado à Teoria da Seleção Natural e aos conceitos de genética, pois nele, inicialmente, cria-se uma população de soluções aleatórias e, dentre elas, por meio da função de avaliação (cálculo do fitness), serão escolhidas aquelas que têm maior fitness para serem reproduzidas. Em seguida, é realizado o que na biologia chama-se de Crossover, troca de material genético entre cromossomos na criação das células filhas. No caso do algoritmo, o que acontece é uma troca entre bits nas soluções a fim de promover uma melhora do fitness para os filhos - seleciona-se a mesma quantidade de bits a 0 e a 1 e troca-se: os que estavam a 1 ficam a 0 e os que estavam a 0 ficam a 1. Isso pode vir a melhorar o fitness da solução uma vez que possuem custos diferentes, portanto, a soma dos novos valores pode ser vantajosa. Estes filhos gerados substituem a população antiga e faz-se o mesmo ciclo até que não haja uma melhora do fitness ou até que o número máximo de gerações seja atingido (condição máxima de paragem).

Para esta importante etapa do Crossover, implementou-se 3 possibilidades de funções para realizar o procedimento. A primeira delas (*crossover*) escolhe um ponto de corte no vetor P e a partir desse ponto a população filha recebe as características do 'Pai 2' e antes do ponto, as características do 'Pai 1'. O mesmo acontece com a segunda população filha, mas de forma inversa. Já a segunda função (*crossover2*) define 2 pontos de corte e o início fica com as características do 'Pai 1', o meio com as características do 'Pai 2' e o fim com o

restante do ‘Pai 1’. E no terceiro método (*crossover_uniforme*), o crossover acontece uniformemente de forma mais agressiva: para cada elemento do vetor, lança-se uma moeda de probabilidade em que pode resultar em características do ‘Pai 1’ ou características do ‘Pai 2’, ou seja, o filho herda de forma aleatória as características dos pais.

Os três métodos geram, entretanto, soluções inválidas. E para resolver este problema, surge a necessidade de integrar na função de avaliação *(evaluate)* um mecanismo de reparação que torna a solução inválida em uma solução válida ao equilibrar o número **m** de elementos na solução. Com esta finalidade, a função percorre o vetor P de elementos da solução e realiza a contagem de quantos estão selecionados (bits a 1). Se este número for menor do que o **m**, seleciona aleatoriamente locais com bit a 0 e coloca-os a 1, até que o número de selecionados seja exatamente igual à **m**. Caso contrário, se a solução estiver com excesso de elementos selecionados, desseleciona aleatoriamente até que cumpra a condição principal da solução. Adicionalmente, implementou-se uma validação final por penalização: se, após a tentativa de reparação, ocorrer algum erro crítico e a solução ainda não tiver exatamente **m** locais, o fitness é penalizado para 0.0. Isto não torna a solução válida, mas garante que esta é descartada pelo processo de seleção natural, assegurando a robustez do algoritmo.

Foram desenvolvidos também 2 métodos para mutação (*mutation* e *mutation2*), um que percorre todas as posições do vetor p, lança um dado de probabilidade e, se o número gerado for menor do que o parâmetro ‘pm’ pré-estabelecido, o bit para este elemento é trocado e ele passa a ser selecionado para a próxima geração ou passa a ser descartado (depende se a troca foi para 1 ou para 0) - isso pode fazer com que a solução tenha mais ou menos elementos do que a quantidade **m** estabelecida, sendo assim importante a passagem pela função *evaluate*; e outro que ao invés de simplesmente trocar o bit de um elemento sem verificar a quantidade **m** de elementos que a solução deve ter, sorteia duas posições aleatórias dentro do vetor, uma que está a 0 e outra que está a 1, e troca o valor entre elas, fazendo com que o número de elementos selecionados seja mantido.

Sobre a questão da seleção dos pais da próxima população filha, implementou-se o Torneio Binário e o Torneio-K. O primeiro método escolhe dois elementos aleatórios da população atual para fazer a comparação do fitness e fica aquele que tem maior qualidade para ser um pai de uma geração. Já o segundo escolhe **k** elementos aleatórios, compara os seus fitness e seleciona aquele que possui maior valor também. A diferença é que neste

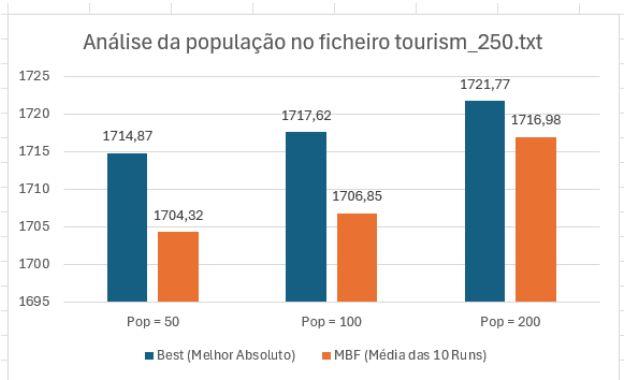
último a pressão seletiva é muito mais alta e fica muito difícil um indivíduo médio ganhar. Entretanto, perde-se muito mais diversidade também.

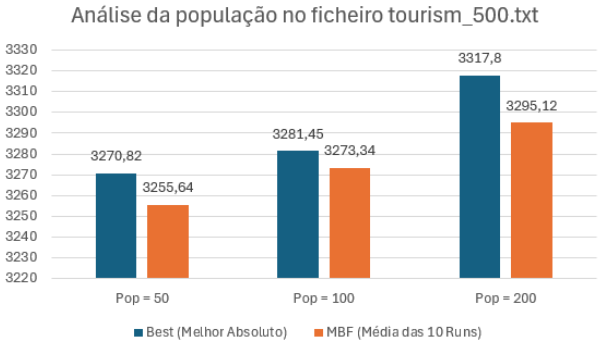
2.2.1 Testes experimentais do algoritmo evolutivo:

A análise foca nas instâncias de maior complexidade (*tourism_250.txt* e *tourism_500.txt*), onde a capacidade de escapar a ótimos locais é crítica.

2.2.2 Impacto do Tamanho da População

Testou-se o algoritmo variando a dimensão da população (50, 100 e 200 indivíduos), mantendo fixos os restantes parâmetros (2500 gerações, Torneio Binário).

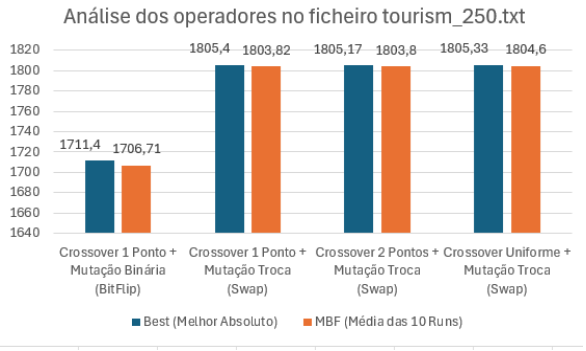


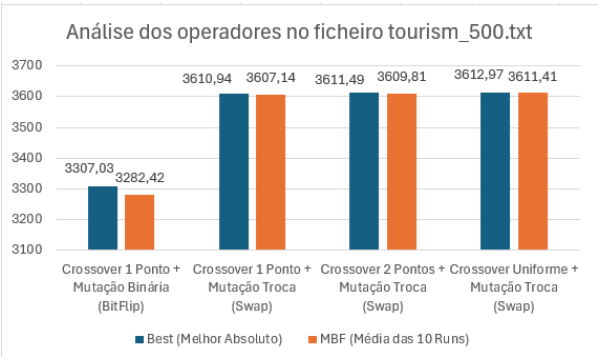


Neste experimento conclui-se que, em ambos os ficheiros, quanto maior a população, mais perto chega-se da solução ótima.

2.2.3. Comparação de Operadores Genéticos

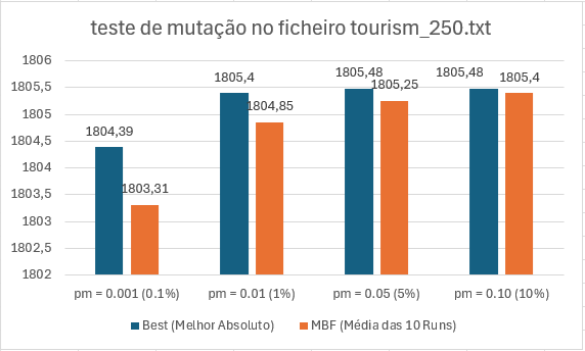
Avaliou-se o desempenho de diferentes estratégias de Crossover e Mutação. Os restantes parâmetros estabeleceram-se fixos e a população foi 100.

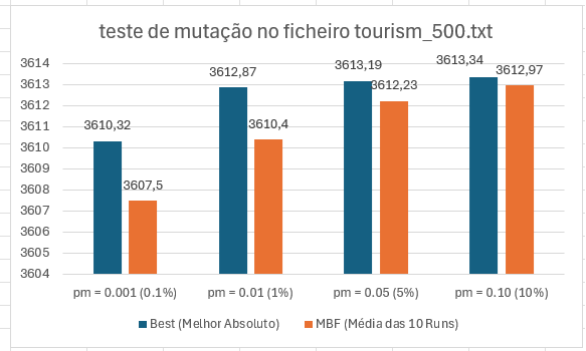


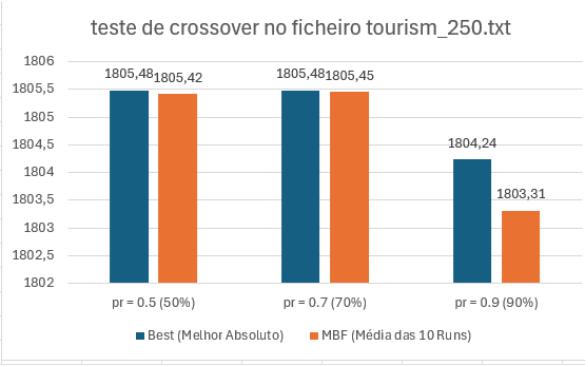


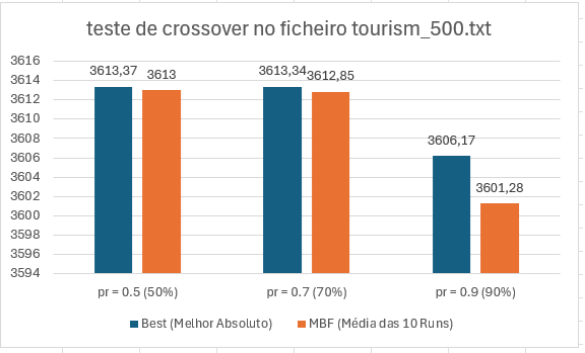
Este experimento conclui que a utilização dos operadores Crossover de 1 ponto com mutação de troca resulta em soluções com resultados significativamente melhores quando comparados ao uso da mutação binária.

2.2.4. Experimento de probabilidades







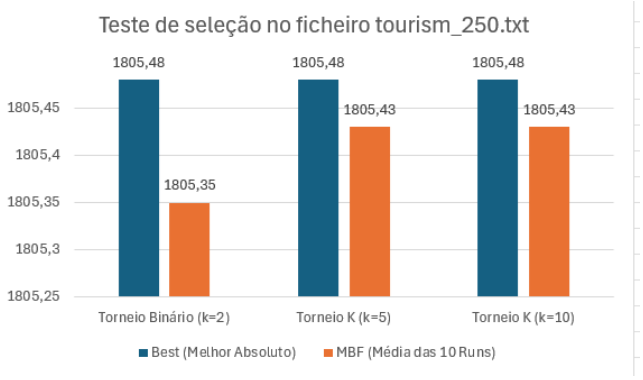


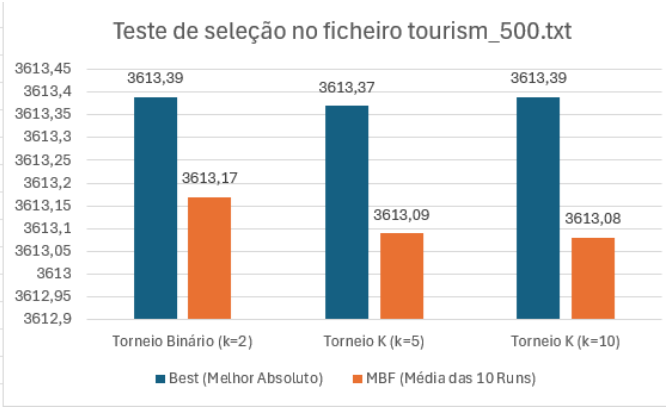
Conclusão relativamente à mutação: Quanto maior a "pm" (probabilidade de mutação), melhor a solução, e isso ocorre em ambos os ficheiros.

Conclusão relativamente ao crossover: Quanto menor a "pr" (probabilidade de recombinação), melhor a solução. No caso do ficheiro *tourism_250.txt* com um pr = 0.7 se obteve uma média melhor relativamente ao uso do pr=0.5, porém o best (melhor absoluto) foi igual, e superior ao teste em que foi utilizado o pr=0.9.

2.2.5. Pressão Seletiva (Torneio)

Por fim, comparou-se o método de seleção: Torneio Binário (k=2) vs. Torneio-K (k=5 ou k=10).





Neste experimento conclui-se que, no ficheiro *tourism_250.txt* os resultados obtidos foram iguais no best (melhor absoluto), entretanto, na média, notou-se que o torneio-k obteve resultados ligeiramente superiores. Já no ficheiro *tourism_500.txt* observou-se que o torneio binário obteve média igual ao torneio-k sendo k=10 mas obteve uma média superior aos dois outros testes que foram feitos a utilizar o torneio-k.

2.3. Algoritmo híbrido:

O híbrido é, então, uma combinação dos dois algoritmos falados anteriormente, Trepa-Colinas e Evolutivo. Traz uma vantagem muito grande, pois adiciona a capacidade do Evolutivo de criar uma população de soluções com boas características à capacidade do Trepa-Colinas de encontrar a melhor solução dentre essas opções.

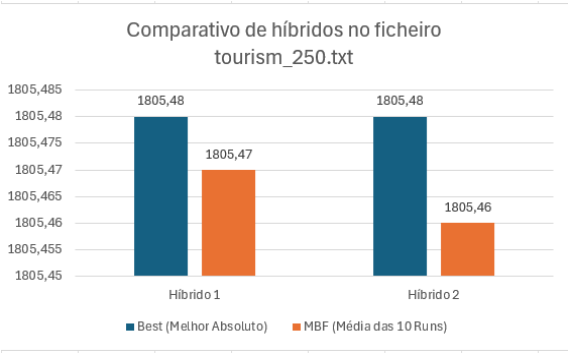
Para este último método foram implementadas duas versões: uma que faz o refinamento da população e outra que faz o refinamento final. O primeiro trabalha com uma abordagem de primeiro, ao invés de começar a evolução com indivíduos aleatórios, aplicar o Trepa-Colinas logo de antemão para que já exista um refinamento inicial da população e a evolução seja feita de forma inteligente, a contemplar somente a melhor versão de cada um dos elementos. Dessa forma, garante-se que o arranque para as próximas gerações seja feito de um patamar superior. Já o segundo método não aplica o Trepa-Colinas a cada uma das soluções, mas sim apenas na melhor solução encontrada unicamente pelo algoritmo

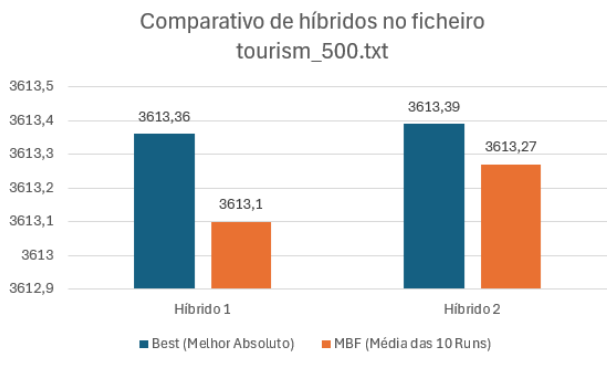
evolutivo. Portanto, primeiro faz-se uma abordagem do evolutivo para todas as gerações (2500) e dentre essas, escolhe-se aquela que tem melhor qualidade para aplicar o Trepa-Colinas e encontrar a sua melhor versão. Isto garante que a solução final seja verdadeiramente ótima.

2.3.1. Testes experimentais ao algoritmo híbrido

Neste teste experimental, foi escolhido os melhores algoritmos encontrados no estudo experimental anterior (*gera_vizinhos*, mutação troca, *crossover2* e torneio binário) para construir as abordagens híbridas (melhor pesquisa local + melhor evolutivo)

2.3.2. O comparativo das abordagens híbridas:





Nestes testes conclui-se que ambas as abordagens híbridas foram bem implementadas, em que as duas obtiveram resultados próximos da solução ótima. No ficheiro *tourism_250.txt* o que obteve melhor média foi o da abordagem 1, já no ficheiro *tourism_500.txt* a abordagem 2 obteve melhor média e também melhor valor absoluto (best).

3. Justificação das principais opções tomadas:

Para a implementação dos algoritmos, a estruturação dos dados foi algo crucial para mantê-los organizados e com código limpo. Para isso, implementamos uma estrutura para cada solução composta por 3 variáveis:

- P: array binário estático com dimensão C (nº de candidaturas) em que as posições assinaladas com 1 são locais selecionados e as assinaladas com 0 são os pontos turísticos descartados da solução;
- Fitness: float que armazena o valor retornado pela função de avaliação;

- Valido: inteiro que sinaliza se a solução respeita as restrições estabelecidas ou não. Isso elimina a necessidade de percorrer o vetor P para conferir tudo novamente e assim isso só se faz uma vez.

As distâncias entre os locais são guardadas numa matriz bidimensional quadrada que pode ser representada da seguinte forma:

	A	B	C
A	0	d(a,b)	d(a,c)
B	d(a,b)	0	d(b,c)
C	d(a,c)	d(b,c)	0

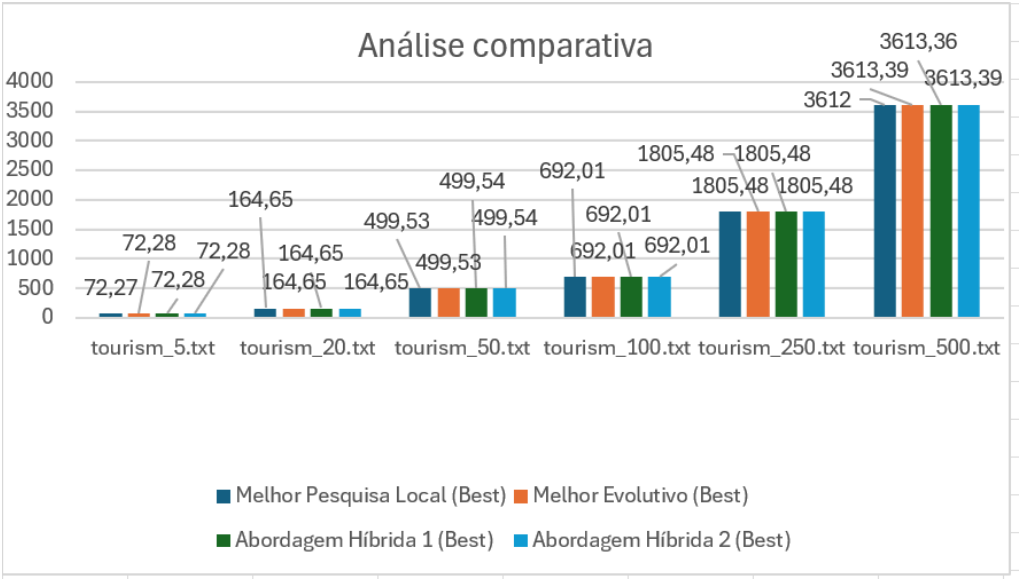
Em termos de código, optou-se por criar esta matriz de forma estática no início da execução. Isto mantém os dados organizados na memória do computador, permitindo que a função de avaliação leia as distâncias muito mais depressa – o que é essencial, dado que esta leitura é repetida milhares de vezes ao longo da evolução.

4. Comparação geral dos experimentos:

Para todos os algoritmos implementados, o aumento do número de iterações foi essencial para obter melhores resultados. Entretanto, para ficheiros pequenos, os algoritmos foram capazes de encontrar o máximo global com um baixo número de iterações.

A seguir, apresenta-se uma tabela comparativa com os resultados obtidos por melhor pesquisa local, melhor evolutivo, abordagem híbrida 1 e abordagem híbrida 2.

Tabela de comparação dos algortimos				
Tamanho Problema	Melhor Pesquisa Local (Best)	Melhor Evolutivo (Best)	Abordagem Híbrida 1 (Best)	Abordagem Híbrida 2 (Best)
tourism_5.txt	72,27	72,28	72,28	72,28
tourism_20.txt	164,65	164,65	164,65	164,65
tourism_50.txt	499,53	499,53	499,54	499,54
tourism_100.txt	692,01	692,01	692,01	692,01
tourism_250.txt	1805,48	1805,48	1805,48	1805,48
tourism_500.txt	3612	3613,39	3613,36	3613,39



Percebe-se que para os menores ficheiros, todos os algoritmos alcançaram resultados quase totalmente iguais, e somente para o maior ficheiro (*tourism_500.txt*) há uma diferença entre os valores obtidos. Para este, a abordagem Híbrida 2, com Trepa-Colinas final, e o Evolutivo mostraram-se mais eficientes em relação aos outros algoritmos.

5. Conclusão

Em relação ao potencial máximo (Best), o algoritmo de Pesquisa Local, apesar de sua simplicidade em relação aos outros, conseguiu isolar a melhor solução para ficheiros de até complexidade média. Contudo, para ficheiros maiores, como o *tourism_500.txt,* ele perde significativamente para os algoritmos Evolutivo e Híbridos, justificando que realmente carece de uma exploração mais global e mais densa a fim de esquivar de soluções sub-ótimas (máximos locais).

Já para a capacidade dos algoritmos de encontrarem uma solução ótima de forma consistente (MBF), os algoritmos Evolutivo e Híbridos apresentam maior eficiência dado os resultados obtidos e mostram-se mais capazes de não ficarem estagnados em máximos locais, como acontece com o algoritmo Trepa-Colinas, estabelecendo uma média de qualidade superior.

Dessa forma, pode-se concluir que os quatro algoritmos são eficientes, cada um com as suas particularidades, mas que, de forma geral, competem bem uns com os outros.

Coimbra
2025